

The Monte Carlo method thus becomes a deterministic quadrature scheme — albeit a simple one — whose fractional error decreases at least as fast as N^{-1} (even faster if the function goes to zero smoothly at the boundaries of the sampled region or is periodic in the region).

The trouble with a grid is that one has to decide *in advance* how fine it should be. One is then committed to completing all of its sample points. With a grid, it is not convenient to “sample *until*” some convergence or termination criterion is met. One might ask if there is not some intermediate scheme, some way to pick sample points “at random,” yet spread out in some self-avoiding way, avoiding the chance clustering that occurs with uniformly random points.

A similar question arises for tasks other than Monte Carlo integration. We might want to search an n -dimensional space for a point where some (locally computable) condition holds. Of course, for the task to be computationally meaningful, there had better be continuity, so that the desired condition will hold in some finite n -dimensional neighborhood. We may not know a priori how large that neighborhood is, however. We want to “sample *until*” the desired point is found, moving smoothly to finer scales with increasing samples. Is there any way to do this that is better than uncorrelated, random samples?

The answer to the above question is “yes.” Sequences of n -tuples that fill n -space more uniformly than uncorrelated random points are called *quasi-random sequences*. That term is somewhat of a misnomer, since there is nothing “random” about quasi-random sequences: They are cleverly crafted to be, in fact, *subrandom*. The sample points in a quasi-random sequence are, in a precise sense, “maximally avoiding” of each other.

A conceptually simple example is *Halton’s sequence* [1]. In one dimension, the j th number H_j in the sequence is obtained by the following steps: (i) Write j as a number in base b , where b is some prime. (For example, $j = 17$ in base $b = 3$ is 122.) (ii) Reverse the digits and put a radix point (i.e., a decimal point base b) in front of the sequence. (In the example, we get 0.221 base 3.) The result is H_j . To get a sequence of n -tuples in n -space, you make each component a Halton sequence with a different prime base b . Typically, the first n primes are used.

It is not hard to see how Halton’s sequence works: Every time the number of digits in j increases by one place, j ’s digit-reversed fraction becomes a factor of b finer-meshed. Thus the process is one of filling in all the points on a sequence of finer and finer Cartesian grids — and in a kind of maximally spread-out order on each grid (since, e.g., the most rapidly changing digit in j controls the *most* significant digit of the fraction).

Other ways of generating quasi-random sequences have been proposed by Faure, Sobol’, Niederreiter, and others. Bratley and Fox [2] provide a good review and references, and discuss a particularly efficient variant of the Sobol’ [3] sequence suggested by Antonov and Saleev [4]. It is this Antonov-Saleev variant whose implementation we now discuss.

The Sobol’ sequence generates numbers between zero and one directly as binary fractions of length w bits, from a set of w special binary fractions, V_i , $i = 1, 2, \dots, w$, called *direction numbers*. In Sobol’ original method, the j th number X_j is generated by XORing (bitwise exclusive or) together the set of V_i ’s satisfying the criterion on i , “the i th bit of j is nonzero.” As j increments, in other words, different ones of the V_i ’s flash in and out of X_j on different time scales. V_1 alternates between being present and absent most quickly, while

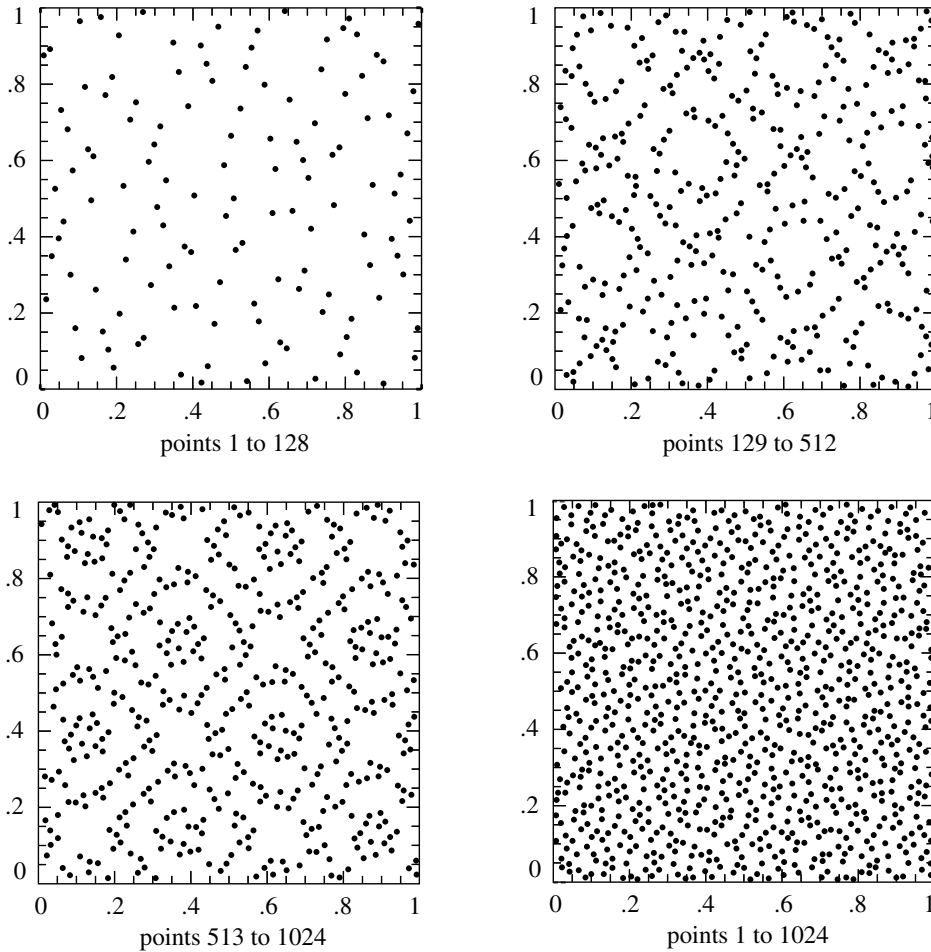


Figure 7.8.1. First 1024 points of a two-dimensional Sobol' sequence. The sequence is generated number-theoretically, rather than randomly, so successive points at any stage “know” how to fill in the gaps in the previously generated distribution.

V_k goes from present to absent (or vice versa) only every 2^{k-1} steps.

Antonov and Saleev's contribution was to show that instead of using the bits of the integer j to select direction numbers, one could just as well use the bits of the *Gray code* of j , $G(j)$. (For a quick review of Gray codes, look at §22.3.)

Now $G(j)$ and $G(j + 1)$ differ in exactly one bit position, namely in the position of the rightmost zero bit in the binary representation of j (adding a leading zero to j if necessary). A consequence is that the $j + 1$ st Sobol'-Antonov-Saleev number can be obtained from the j th by XORing it with a *single* V_i , namely with i the position of the rightmost zero bit in j . This makes the calculation of the sequence very efficient, as we shall see.

Figure 7.8.1 plots the first 1024 points generated by a two-dimensional Sobol' sequence. One sees that successive points do “know” about the gaps left previously, and keep filling them in, hierarchically.

We have deferred to this point a discussion of how the direction numbers V_i are generated. Some nontrivial mathematics is involved in that, so we will content ourselves with a cookbook summary only: Each different Sobol' sequence (or component of an n -dimensional

Degree	Primitive Polynomials Modulo 2*
1	0 (i.e., $x + 1$)
2	1 (i.e., $x^2 + x + 1$)
3	1, 2 (i.e., $x^3 + x + 1$ and $x^3 + x^2 + 1$)
4	1, 4 (i.e., $x^4 + x + 1$ and $x^4 + x^3 + 1$)
5	2, 4, 7, 11, 13, 14
6	1, 13, 16, 19, 22, 25
7	1, 4, 7, 8, 14, 19, 21, 28, 31, 32, 37, 41, 42, 50, 55, 56, 59, 62
8	14, 21, 22, 38, 47, 49, 50, 52, 56, 67, 70, 84, 97, 103, 115, 122
9	8, 13, 16, 22, 25, 44, 47, 52, 55, 59, 62, 67, 74, 81, 82, 87, 91, 94, 103, 104, 109, 122, 124, 137, 138, 143, 145, 152, 157, 167, 173, 176, 181, 182, 185, 191, 194, 199, 218, 220, 227, 229, 230, 234, 236, 241, 244, 253
10	4, 13, 19, 22, 50, 55, 64, 69, 98, 107, 115, 121, 127, 134, 140, 145, 152, 158, 161, 171, 181, 194, 199, 203, 208, 227, 242, 251, 253, 265, 266, 274, 283, 289, 295, 301, 316, 319, 324, 346, 352, 361, 367, 382, 395, 398, 400, 412, 419, 422, 426, 428, 433, 446, 454, 457, 472, 493, 505, 508
*Expressed as a decimal integer whose binary representation gives the coefficients, from the highest to lowest power of x . Only the internal terms are represented — the highest-order term and the constant term always have coefficient 1.	

sequence) is based on a different primitive polynomial over the integers modulo 2, that is, a polynomial whose coefficients are either 0 or 1, and which generates a maximal length shift register sequence. (Primitive polynomials modulo 2 were used in §7.5 and are further discussed in §22.4.) Suppose P is such a polynomial, of degree q ,

$$P = x^q + a_1x^{q-1} + a_2x^{q-2} + \cdots + a_{q-1}x + 1 \quad (7.8.1)$$

Define a sequence of integers M_i by the q -term recurrence relation,

$$M_i = 2a_1M_{i-1} \oplus 2^2a_2M_{i-2} \oplus \cdots \oplus 2^{q-1}M_{i-q+1}a_{q-1} \oplus (2^qM_{i-q} \oplus M_{i-q}) \quad (7.8.2)$$

Here bitwise XOR is denoted by $\oplus$. The starting values for this recurrence are that $M_1, \dots, M_q$ can be arbitrary odd integers less than $2, \dots, 2^q$, respectively. Then, the direction numbers V_i are given by

$$V_i = M_i / 2^i \quad i = 1, \dots, w \quad (7.8.3)$$

The table above lists all primitive polynomials modulo 2 with degree $q \leq 10$. Since the coefficients are either 0 or 1, and since the coefficients of x^q and of 1 are predictably 1, it is convenient to denote a polynomial by its middle coefficients taken as the bits of a binary number (higher powers of x being more significant bits). The table uses this convention.

Turn now to the implementation of the Sobol' sequence. Successive calls to the function `sobseq` (after a preliminary initializing call) return successive points in an n -dimensional Sobol' sequence based on the first n primitive polynomials in the table. As given, the routine is initialized for maximum n of 6 dimensions, and for a word length w of 30 bits. These parameters can be altered by changing `MAXBIT` ($\equiv w$) and `MAXDIM`, and by adding more initializing data to the arrays `ip` (the primitive polynomials from the table above), `mdeg` (their degrees), and `iv` (the starting values for the recurrence, equation 7.8.2). A second table, on the next page, elucidates the initializing data in the routine.